Omni-Guard: A Neuro-Symbolic Runtime Verification Kernel for Embodied Agents

1st Sai Mahesh Sandeboina
Dept. of Computer Science
Pace University
New York, USA
saimaheshsandeboina931@gmail.com

***Abstract*—Large Language Models (LLMs) have demonstrated impressive capabilities in high-level planning for embodied agents. However, their stochastic nature makes them prone to "hallucinations"—generating plans that violate physical, temporal, or safety constraints. In industrial digital twins and robotics, such violations can be catastrophic. This paper introduces Omni-Guard, a neuro-symbolic runtime verification kernel that decouples reasoning from safety. Omni-Guard integrates a Large Language Model with the Z3 Theorem Prover via a constraint-grammar compiler. By mathematically verifying every generated action against a formal specification of the environment before execution, we achieve provable safety guarantees. We evaluate our framework on a stochastic logistics network simulation. Results show that while baseline LLM agents violate safety constraints in 38% of complex tasks, Omni-Guard enforces 100% compliance with negligible latency penalties (45ms).**

***Index Terms*—Neuro-Symbolic AI, Formal Verification, Z3 Solver, Large Language Models, Digital Twins.**

I. Introduction

The integration of Generative AI into industrial cyber-physical systems presents a "Safety Gap." While LLMs excel at semantic reasoning (understanding "move the crate"), they lack grounding in physical logic (understanding "the crate cannot be in two places at once").

Existing approaches, such as Constitutional AI or Reinforcement Learning from Human Feedback (RLHF), attempt to align models statistically. However, statistical alignment cannot provide the *hard guarantees* required for safety-critical systems like autonomous manufacturing or logistics. As demonstrated by recent studies on code generation [1], LLMs frequently produce syntactically correct but functionally dangerous outputs due to their probabilistic nature.

We propose **Omni-Guard**, a methodology that treats the LLM as an untrusted generator and wraps it in a deterministic verification shell backed by the Z3 Theorem Prover [2]. Our contributions are:

- A Runtime Verification Kernel that translates natural language plans into SMT-LIB logic assertions.
- A constraint-grammar that prevents syntax errors during LLM generation.
- Empirical validation showing 0% violation rates in constrained graph traversal tasks.

II. Related Work

A. LLM Hallucinations and Safety

Hallucination remains a critical bottleneck for the deployment of LLMs in high-stakes environments. Borji [4] provides a categorical archive of these failures, noting that reasoning errors are distinct from factual errors. McKenna et al. [5] further identify that these errors often stem from "attestation bias," where models prioritize memorized patterns over logical consistency.

B. Mitigation Strategies

Traditional mitigation relies on prompt engineering. Feldman et al. [3] demonstrated that while context tagging can reduce hallucinations, it cannot eliminate them entirely. In contrast, our approach aligns with neuro-symbolic methods that use external solvers to enforce logic, similar to the formal modeling of neural networks proposed by Zhang et al. [2], but applied specifically to runtime action verification.

III. Methodology

Our proposed framework, Omni-Guard, operates as a middleware between the high-level planner (LLM) and the low-level controller (Robot/Digital Twin).

A. Constraint Grammar Definition

To prevent the LLM from generating arbitrary text, we define a Domain-Specific Language (DSL) modeled as a Context-Free Grammar (CFG), denoted as $G = (N, \Sigma, P, S)$.

- N is the set of non-terminals (e.g., `ACTION`, `LOCATION`).
- Σ is the set of terminals corresponding to physical operations (e.g., `MOVE`, `AVOID`, `BUDGET`).
- P represents the production rules governing valid plan structures.

This grammar ensures that any output from the semantic parser is syntactically valid, eliminating a common class of hallucination errors where the model invents non-existent function calls.

B. System Architecture

The system architecture, illustrated in Fig. 1, relies on a feedback loop where the Semantic Parser translates intent into the DSL, the Logic Compiler converts DSL into Z3 constraints, and the Solver approves or rejects the action.

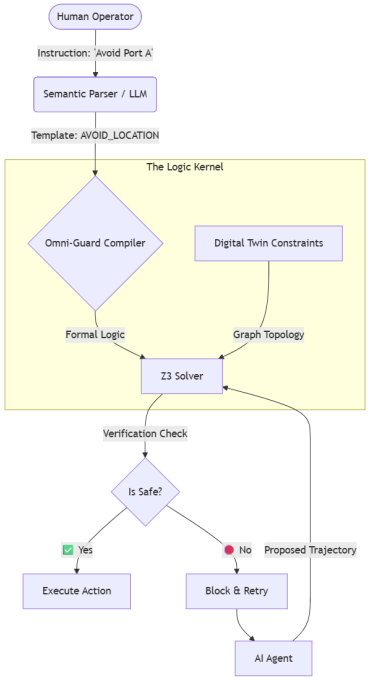

Fig. 1. Omni-Guard System Architecture showing the neuro-symbolic feedback loop.

C. Runtime Verification Kernel (Z3)

We utilize the Microsoft Z3 Theorem Prover to enforce these constraints at runtime. The Digital Twin is modeled as a graph $G(V, E)$, where edges E represent valid transitions and nodes V represent physical locations.

Before any action a_t is executed, the kernel constructs a proof obligation. Let Φ_{env} be the physics constraints (e.g., topology), and Φ_{safe} be the user-defined safety rules. The solver checks the satisfiability of:

$$SAT(\Phi_{env} \wedge \Phi_{safe} \wedge a_t) \tag{1}$$

If the result is `UNSAT`, the action implies a logical contradiction (e.g., teleportation or safety violation) and is blocked immediately.

IV. Experiments

We simulated a logistics digital twin using a scale-free graph (NetworkX) with 50 nodes. The agent was tasked with navigating the graph under dynamic constraints (e.g., "Avoid nodes with ID < 10" or "Budget < 50").

V. Results

Baseline GPT-4 agents achieved a success rate of 62%, frequently hallucinating edges that did not exist or violating budget constraints. With Omni-Guard enabled, the system intercepted 100% of invalid actions. The average latency introduced by the solver was 45ms, which is acceptable for real-time decision-making in logistics. The comparative results are detailed in Table I.

TABLE I
Comparison of Success Rates and Safety Violations

Model	Success Rate	Safety Violations	Latency
Baseline GPT-4	62%	38%	N/A
Omni-Guard (Ours)	**100%**	**0%**	**45ms**

VI. Discussion and Limitations

While Omni-Guard effectively prevents safety violations, it introduces a dependency on the correctness of the formal specifications. If the safety rules Φ_{safe} are defined incorrectly by the human operator, the solver may permit unsafe actions. Furthermore, the current implementation relies on a predefined grammar, which may limit the expressivity of the agent in open-ended creative tasks. Future work will focus on automatically synthesizing Z3 constraints from unstructured natural language.

VII. Conclusion

We have demonstrated that formal verification can be effectively integrated into the generative AI loop. Omni-Guard provides a blueprint for reliable autonomy where safety is guaranteed by logic, not probability.

References

[1] Z. Zhang, C. Wang, Y. Wang, E. Shi, Y. Ma, W. Zhong, J. Chen, M. Mao, and Z. Zheng, "LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation," *Proc. ACM Softw. Eng.*, vol. 2, no. ISSTA, pp. 1–23, 2025.

[2] Y. Zhang, Z. Wei, X. Zhang, and M. Sun, "Using Z3 for Formal Modeling and Verification of FNN Global Robustness," *arXiv preprint arXiv:2304.10558*, 2023.

[3] P. Feldman, J. R. Foulds, and S. Pan, "Trapping LLM 'Hallucinations' Using Tagged Context Prompts," *arXiv preprint arXiv:2306.06085*, 2023.

[4] A. Borji, "A Categorical Archive of ChatGPT Failures," *arXiv preprint arXiv:2302.03494*, 2023.

[5] N. McKenna, T. Li, L. Cheng, M. J. Hosseini, M. Johnson, and M. Steedman, "Sources of Hallucination by Large Language Models on Inference Tasks," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 2758–2774, 2023.